Advanced Java Programming

Assignment No. 1 – Unit 1: Networking & Database (JDBC)

Q1. Java program using InetAddress class to display the IP address of a given host name.

Concept:

The **InetAddress** class in **java.net** package represents an IP address. It provides methods to resolve host names to IP addresses and vice versa.

Program:

```
import java.net.InetAddress;
import java.net.UnknownHostException;

public class InetAddressDemo {
    public static void main(String[] args) {
        try {
            // Get InetAddress for a hostname
            InetAddress address = InetAddress.getByName("www.google.com");

            System.out.println("Host Name : " + address.getHostName());
            System.out.println("IP Address: " + address.getHostAddress());

            // Get local host info
            InetAddress localHost = InetAddress.getLocalHost();
            System.out.println("\nLocal Host Name : " + localHost.getHostName());
            System.out.println("Local IP Address: " + localHost.getHostAddress());

            // Get all IP addresses for a host
            InetAddress[] allAddresses = InetAddress.getAllByName("www.google.com");
            System.out.println("\nAll IP Addresses for www.google.com:");
            for (InetAddress addr : allAddresses) {
                System.out.println("  " + addr.getHostAddress());
            }

        } catch (UnknownHostException e) {
            System.out.println("Host not found: " + e.getMessage());
        }
    }
}
```

Output:

```
Host Name : www.google.com
IP Address: 142.250.195.100

Local Host Name : DESKTOP-ABC
Local IP Address: 192.168.1.10

All IP Addresses for www.google.com:
  142.250.195.100
  142.250.195.101
```

■ *Note: InetAddress.getByName() throws UnknownHostException if host not found. Always handle it.*

Q2. Java program using URL class to read and display different components of a given URL.

Concept:

The **URL** class in java.net package represents a Uniform Resource Locator. It provides methods to extract individual components like protocol, host, port, path, query, and reference.

Program:

```
import java.net.URL;
import java.net.MalformedURLException;

public class URLDemo {
    public static void main(String[] args) {
        try {
            URL url = new URL("https://www.example.com:8080/path/page.html?name=Java&ver=17#section1");

            System.out.println("Full URL     : " + url.toString());
            System.out.println("Protocol     : " + url.getProtocol());
            System.out.println("Host         : " + url.getHost());
            System.out.println("Port         : " + url.getPort());
            System.out.println("Default Port : " + url.getDefaultPort());
            System.out.println("Path (File)  : " + url.getFile());
            System.out.println("Path Only    : " + url.getPath());
            System.out.println("Query String : " + url.getQuery());
            System.out.println("Reference    : " + url.getRef());
            System.out.println("Authority    : " + url.getAuthority());
            System.out.println("User Info    : " + url.getUserInfo());

        } catch (MalformedURLException e) {
            System.out.println("Invalid URL: " + e.getMessage());
        }
    }
}
```

Output:

```
Full URL     : https://www.example.com:8080/path/page.html?name=Java&ver=17#section1
Protocol     : https
Host         : www.example.com
Port         : 8080
Default Port : 443
Path (File)  : /path/page.html?name=Java&ver=17
Path Only    : /path/page.html
Query String : name=Java&ver=17
Reference    : section1
Authority    : www.example.com:8080
User Info    : null
```

Q3. Java program using Socket and ServerSocket to implement one-way communication between client and server.

Concept:

A **ServerSocket** listens on a port for incoming connections. A **Socket** is used by the client to connect. One-way means only the client sends messages to the server.

Server Program (Server.java):

```
import java.net.*;
import java.io.*;

public class Server {
    public static void main(String[] args) throws IOException {
        ServerSocket serverSocket = new ServerSocket(5000);
        System.out.println("Server started. Waiting for client...");

        Socket socket = serverSocket.accept();
        System.out.println("Client connected: " + socket.getInetAddress());

        // Read from client
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));

        String message;
        System.out.println("Messages from client:");
```

```
        while ((message = in.readLine()) != null) {
            System.out.println("Client: " + message);
        }

        in.close();
        socket.close();
        serverSocket.close();
        System.out.println("Connection closed.");
    }
}
```

Client Program (Client.java):

```
import java.net.*;
import java.io.*;

public class Client {
    public static void main(String[] args) throws IOException {
        Socket socket = new Socket("localhost", 5000);
        System.out.println("Connected to server.");

        // Send messages to server
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        out.println("Hello Server!");
        out.println("This is one-way communication.");
        out.println("Goodbye!");

        out.close();
        socket.close();
        System.out.println("Messages sent. Connection closed.");
    }
}
```

Output (Server side):

```
Server started. Waiting for client...
Client connected: /127.0.0.1
Messages from client:
Client: Hello Server!
Client: This is one-way communication.
Client: Goodbye!
Connection closed.
```

■ *Note: Run Server.java first, then Client.java. Server uses accept() which is a blocking call.*

Q4. Java program using java.net package to implement a multi-client server that handles client requests using sockets and streams.

Concept:

Multi-client server uses **Threads** — each client connection is handled in a separate thread so multiple clients can connect simultaneously.

Multi-Client Server (MultiServer.java):

```
import java.net.*;
import java.io.*;

// Thread to handle each client
class ClientHandler extends Thread {
    private Socket socket;
    private int clientId;

    public ClientHandler(Socket socket, int id) {
        this.socket = socket;
        this.clientId = id;
    }

    public void run() {
```

```
        try {
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

            out.println("Welcome Client-" + clientId + "! Type your message:");

            String message;
            while ((message = in.readLine()) != null) {
                System.out.println("Client-" + clientId + ": " + message);
                out.println("Server Echo to Client-" + clientId + ": " + message);
                if (message.equalsIgnoreCase("bye")) break;
            }

            in.close();
            out.close();
            socket.close();
            System.out.println("Client-" + clientId + " disconnected.");
        } catch (IOException e) {
            System.out.println("Client-" + clientId + " error: " + e.getMessage());
        }
    }
}

public class MultiServer {
    public static void main(String[] args) throws IOException {
        ServerSocket serverSocket = new ServerSocket(6000);
        System.out.println("Multi-Client Server started on port 6000...");
        int clientCount = 0;

        while (true) {
            Socket clientSocket = serverSocket.accept();
            clientCount++;
            System.out.println("Client-" + clientCount + " connected.");
            ClientHandler handler = new ClientHandler(clientSocket, clientCount);
            handler.start();  // Start new thread for this client
        }
    }
}
```

Client Program (MultiClient.java):

```
import java.net.*;
import java.io.*;
import java.util.Scanner;

public class MultiClient {
    public static void main(String[] args) throws IOException {
        Socket socket = new Socket("localhost", 6000);
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        Scanner sc = new Scanner(System.in);

        System.out.println(in.readLine()); // Welcome message
        String msg;
        while (sc.hasNextLine()) {
            msg = sc.nextLine();
            out.println(msg);
            System.out.println(in.readLine());
            if (msg.equalsIgnoreCase("bye")) break;
        }
        socket.close();
    }
}
```

■ *Note: Open multiple terminals and run MultiClient.java in each. All clients are handled concurrently using threads.*

Q5. Simple two-way chat application using Socket and ServerSocket where both sides can send and receive messages until either types 'exit'.

Concept:

Two-way chat requires both client and server to **read and write** alternately. Each side waits to receive, then sends a reply.

Chat Server (ChatServer.java):

```
import java.net.*;
import java.io.*;
import java.util.Scanner;

public class ChatServer {
    public static void main(String[] args) throws IOException {
        ServerSocket serverSocket = new ServerSocket(7000);
        System.out.println("Chat Server started. Waiting for client...");
        Socket socket = serverSocket.accept();
        System.out.println("Client connected!");

        BufferedReader in   = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        PrintWriter   out   = new PrintWriter(socket.getOutputStream(), true);
        Scanner       sc    = new Scanner(System.in);

        String received, toSend;
        System.out.println("=== Chat Started (type 'exit' to quit) ===");

        while (true) {
            // Receive from client
            received = in.readLine();
            if (received == null || received.equalsIgnoreCase("exit")) {
                System.out.println("Client has left the chat.");
                break;
            }
            System.out.println("Client: " + received);

            // Send to client
            System.out.print("Server: ");
            toSend = sc.nextLine();
            out.println(toSend);
            if (toSend.equalsIgnoreCase("exit")) break;
        }

        in.close(); out.close();
        socket.close(); serverSocket.close();
        System.out.println("Chat ended.");
    }
}
```

Chat Client (ChatClient.java):

```
import java.net.*;
import java.io.*;
import java.util.Scanner;

public class ChatClient {
    public static void main(String[] args) throws IOException {
        Socket socket = new Socket("localhost", 7000);
        System.out.println("Connected to Chat Server!");

        BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        PrintWriter  out  = new PrintWriter(socket.getOutputStream(), true);
        Scanner      sc   = new Scanner(System.in);
```

```
        String toSend, received;
        System.out.println("=== Chat Started (type 'exit' to quit) ===");

        while (true) {
            // Send to server
            System.out.print("Client: ");
            toSend = sc.nextLine();
            out.println(toSend);
            if (toSend.equalsIgnoreCase("exit")) break;

            // Receive from server
            received = in.readLine();
            if (received == null || received.equalsIgnoreCase("exit")) {
                System.out.println("Server has left the chat.");
                break;
            }
            System.out.println("Server: " + received);
        }

        in.close(); out.close(); socket.close();
        System.out.println("Chat ended.");
    }
}
```

Sample Chat Output:

```
Client side:                    Server side:
Client: Hello Server!           Client: Hello Server!
Server: Hi Client! How are you? Server: Hi Client! How are you?
Client: I am fine, thanks!      Client: I am fine, thanks!
Server: Great to hear!          Server: Great to hear!
Client: exit                    Client has left the chat.
Chat ended.                     Chat ended.
```

■■ JDBC (Database Connectivity) Programs ■■

Q6. Java program to connect to a database using JDBC and display all records from a table using Statement and ResultSet.

Concept:

JDBC (Java Database Connectivity) provides API to connect Java programs to databases. Steps: Load driver → Get connection → Create statement → Execute query → Process ResultSet.

Program:

```
import java.sql.*;

public class DisplayRecords {
    // Database credentials
    static final String URL      = "jdbc:mysql://localhost:3306/college_db";
    static final String USER     = "root";
    static final String PASSWORD = "password";

    public static void main(String[] args) {
        Connection conn = null;
        Statement  stmt = null;
        ResultSet  rs   = null;

        try {
            // Step 1: Load and register JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            System.out.println("Driver loaded successfully.");

            // Step 2: Open a connection
            conn = DriverManager.getConnection(URL, USER, PASSWORD);
            System.out.println("Connected to database!");

            // Step 3: Create a Statement object
            stmt = conn.createStatement();

            // Step 4: Execute a query
            String sql = "SELECT * FROM students";
            rs = stmt.executeQuery(sql);

            // Step 5: Extract data from ResultSet
            System.out.println("\n+----+------------------+-----+----------+");
            System.out.println("| ID | Name             | Age | Course   |");
            System.out.println("+----+------------------+-----+----------+");

            while (rs.next()) {
                int    id     = rs.getInt("id");
                String name   = rs.getString("name");
                int    age    = rs.getInt("age");
                String course = rs.getString("course");
                System.out.printf("| %-2d | %-16s | %-3d | %-8s |%n",
                    id, name, age, course);
            }
            System.out.println("+----+------------------+-----+----------+");

        } catch (ClassNotFoundException e) {
            System.out.println("Driver not found: " + e.getMessage());
        } catch (SQLException e) {
            System.out.println("SQL Error: " + e.getMessage());
        } finally {
            // Step 6: Close resources
            try {
                if (rs   != null) rs.close();
                if (stmt != null) stmt.close();
```

```
                if (conn != null) conn.close();
                System.out.println("Connection closed.");
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```

SQL Table Setup:

```
CREATE TABLE students (
    id     INT PRIMARY KEY AUTO_INCREMENT,
    name   VARCHAR(50),
    age    INT,
    course VARCHAR(30)
);
INSERT INTO students VALUES (1,'Rahul Sharma',20,'BCA');
INSERT INTO students VALUES (2,'Priya Patil',21,'MCA');
INSERT INTO students VALUES (3,'Amit Joshi',22,'BSc CS');
```

Output:

```
Driver loaded successfully.
Connected to database!

+----+------------------+-----+----------+
| ID | Name             | Age | Course   |
+----+------------------+-----+----------+
| 1  | Rahul Sharma     | 20  | BCA      |
| 2  | Priya Patil      | 21  | MCA      |
| 3  | Amit Joshi       | 22  | BSc CS   |
+----+------------------+-----+----------+
Connection closed.
```

Q7. Java program using JDBC to insert, update, and delete records in a database table using Statement objects with proper exception handling.

Program:

```
import java.sql.*;

public class InsertUpdateDelete {
    static final String URL  = "jdbc:mysql://localhost:3306/college_db";
    static final String USER = "root";
    static final String PASS = "password";

    public static void main(String[] args) {
        Connection conn = null;
        Statement  stmt = null;

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            conn = DriverManager.getConnection(URL, USER, PASS);
            stmt = conn.createStatement();

            // ■■ INSERT ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            String insertSQL = "INSERT INTO students(name, age, course) " +
                               "VALUES ('Neha Kulkarni', 22, 'MCA')";
            int insertedRows = stmt.executeUpdate(insertSQL);
            System.out.println("INSERT: " + insertedRows + " row(s) inserted.");

            // ■■ UPDATE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            String updateSQL = "UPDATE students SET age = 23 " +
                               "WHERE name = 'Neha Kulkarni'";
            int updatedRows = stmt.executeUpdate(updateSQL);
            System.out.println("UPDATE: " + updatedRows + " row(s) updated.");
```

```
            // ■■ DELETE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            String deleteSQL = "DELETE FROM students WHERE name = 'Neha Kulkarni'";
            int deletedRows = stmt.executeUpdate(deleteSQL);
            System.out.println("DELETE: " + deletedRows + " row(s) deleted.");

        } catch (ClassNotFoundException e) {
            System.out.println("Driver Error: " + e.getMessage());
        } catch (SQLException e) {
            System.out.println("SQL Error Code   : " + e.getErrorCode());
            System.out.println("SQL State        : " + e.getSQLState());
            System.out.println("Error Message    : " + e.getMessage());
        } finally {
            try {
                if (stmt != null) stmt.close();
                if (conn != null) conn.close();
            } catch (SQLException e) { e.printStackTrace(); }
        }
    }
}
```

Output:

```
INSERT: 1 row(s) inserted.
UPDATE: 1 row(s) updated.
DELETE: 1 row(s) deleted.
```

■ *Note: executeUpdate() returns the number of rows affected. Use it for INSERT, UPDATE, DELETE. Use executeQuery() only for SELECT.*

Q8. Java program using PreparedStatement to insert and retrieve records from a database and demonstrate ResultSet navigation.

Concept:

PreparedStatement is precompiled SQL — faster, safer (prevents SQL injection), and supports parameterized queries using '?' placeholders. For navigation, use **TYPE_SCROLL_INSENSITIVE** and **CONCUR_READ_ONLY**.

Program:

```
import java.sql.*;

public class PreparedStmtDemo {
    static final String URL  = "jdbc:mysql://localhost:3306/college_db";
    static final String USER = "root";
    static final String PASS = "password";

    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn = DriverManager.getConnection(URL, USER, PASS);

        // ■■ INSERT using PreparedStatement ■■■■■■■■■■■■■■■■■■■■
        String insertSQL = "INSERT INTO students(name, age, course) VALUES (?, ?, ?)";
        PreparedStatement pstmt = conn.prepareStatement(insertSQL);

        // Insert first record
        pstmt.setString(1, "Sneha Desai");
        pstmt.setInt(2, 20);
        pstmt.setString(3, "BSc CS");
        pstmt.executeUpdate();
        System.out.println("Record 1 inserted.");

        // Insert second record (reuse same PreparedStatement)
        pstmt.setString(1, "Ravi More");
        pstmt.setInt(2, 21);
        pstmt.setString(3, "BCA");
        pstmt.executeUpdate();
        System.out.println("Record 2 inserted.");
```

```
        pstmt.close();

        // ■■ RETRIEVE with Scrollable ResultSet ■■■■■■■■■■■■■■■■
        Statement stmt = conn.createStatement(
            ResultSet.TYPE_SCROLL_INSENSITIVE,
            ResultSet.CONCUR_READ_ONLY);
        ResultSet rs = stmt.executeQuery("SELECT * FROM students");

        // Move to last row
        rs.last();
        System.out.println("\nLAST row  -> ID: " + rs.getInt("id") +
            ", Name: " + rs.getString("name"));

        // Move to first row
        rs.first();
        System.out.println("FIRST row -> ID: " + rs.getInt("id") +
            ", Name: " + rs.getString("name"));

        // Move to specific row (absolute)
        rs.absolute(2);
        System.out.println("ROW 2     -> ID: " + rs.getInt("id") +
            ", Name: " + rs.getString("name"));

        // Forward iteration
        System.out.println("\nAll Records (forward):");
        rs.beforeFirst();
        while (rs.next()) {
            System.out.println("  ID: " + rs.getInt("id") +
                " | Name: " + rs.getString("name") +
                " | Age: "  + rs.getInt("age") +
                " | Course: " + rs.getString("course"));
        }

        // Backward iteration
        System.out.println("\nAll Records (backward - using previous()):");
        rs.afterLast();
        while (rs.previous()) {
            System.out.println("  ID: " + rs.getInt("id") +
                " | Name: " + rs.getString("name"));
        }

        rs.close(); stmt.close(); conn.close();
    }
}
```

Output:

```
Record 1 inserted.
Record 2 inserted.

LAST row  -> ID: 5, Name: Ravi More
FIRST row -> ID: 1, Name: Rahul Sharma
ROW 2     -> ID: 2, Name: Priya Patil

All Records (forward):
  ID: 1 | Name: Rahul Sharma | Age: 20 | Course: BCA
  ID: 2 | Name: Priya Patil  | Age: 21 | Course: MCA
  ...

All Records (backward - using previous()):
  ID: 5 | Name: Ravi More
  ID: 4 | Name: Sneha Desai
  ...
```

Q9. Java program using JDBC to execute CRUD operations using PreparedStatement, and call a stored procedure using CallableStatement with proper exception handling.

Stored Procedure (MySQL):

```
DELIMITER //
CREATE PROCEDURE GetStudentById(IN sid INT)
BEGIN
    SELECT * FROM students WHERE id = sid;
END //
DELIMITER ;
```

Java Program:

```
import java.sql.*;

public class CRUDAndCallable {
    static final String URL  = "jdbc:mysql://localhost:3306/college_db";
    static final String USER = "root";
    static final String PASS = "password";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
            Class.forName("com.mysql.cj.jdbc.Driver");

            // ■■ CREATE (INSERT) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            String insertSQL = "INSERT INTO students(name,age,course) VALUES(?,?,?)";
            try (PreparedStatement ps = conn.prepareStatement(insertSQL,
                    Statement.RETURN_GENERATED_KEYS)) {
                ps.setString(1, "Kiran Jadhav");
                ps.setInt(2, 22);
                ps.setString(3, "MCA");
                ps.executeUpdate();
                ResultSet keys = ps.getGeneratedKeys();
                if (keys.next())
                    System.out.println("Inserted with ID: " + keys.getInt(1));
            }

            // ■■ READ (SELECT) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            String selectSQL = "SELECT * FROM students WHERE course = ?";
            try (PreparedStatement ps = conn.prepareStatement(selectSQL)) {
                ps.setString(1, "MCA");
                ResultSet rs = ps.executeQuery();
                System.out.println("\nMCA Students:");
                while (rs.next())
                    System.out.println("  " + rs.getInt("id") +
                        " - " + rs.getString("name"));
            }

            // ■■ UPDATE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            String updateSQL = "UPDATE students SET age=? WHERE name=?";
            try (PreparedStatement ps = conn.prepareStatement(updateSQL)) {
                ps.setInt(1, 24);
                ps.setString(2, "Kiran Jadhav");
                int rows = ps.executeUpdate();
                System.out.println("\nUpdated rows: " + rows);
            }

            // ■■ DELETE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            String deleteSQL = "DELETE FROM students WHERE name=?";
            try (PreparedStatement ps = conn.prepareStatement(deleteSQL)) {
                ps.setString(1, "Kiran Jadhav");
                int rows = ps.executeUpdate();
                System.out.println("Deleted rows: " + rows);
            }

            // ■■ CallableStatement ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
            System.out.println("\nCalling stored procedure:");
            try (CallableStatement cs = conn.prepareCall("{call GetStudentById(?)}")) {
                cs.setInt(1, 1);
                ResultSet rs = cs.executeQuery();
                while (rs.next())
                    System.out.println("Student: " + rs.getString("name") +
                        ", Age: " + rs.getInt("age"));
            }

        } catch (ClassNotFoundException e) {
            System.out.println("Driver not found: " + e.getMessage());
        } catch (SQLException e) {
            System.out.println("SQL Error [" + e.getErrorCode() + "]: " + e.getMessage());
        }
    }
}
```

Q10. Java program to establish JDBC connection and display all records using Statement and ResultSet.

■ *Note: This is similar to Q6. Below is a concise version with the key JDBC connection steps highlighted.*

```
import java.sql.*;
public class DisplayAll {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college_db", "root", "password");

        Statement stmt = conn.createStatement();
        ResultSet rs   = stmt.executeQuery("SELECT * FROM students");

        System.out.println("ID  | Name              | Age | Course");
        System.out.println("----+-------------------+-----+--------");
        while (rs.next())
            System.out.printf("%-3d | %-17s | %-3d | %s%n",
                rs.getInt("id"), rs.getString("name"),
                rs.getInt("age"), rs.getString("course"));

        rs.close(); stmt.close(); conn.close();
    }
}
```

Q11. Java program using JDBC to insert records into a database table using a Statement object.

```
import java.sql.*;
public class InsertRecord {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college_db", "root", "password");
        Statement stmt = conn.createStatement();

        // Insert a record
        String sql = "INSERT INTO students(name, age, course) " +
                     "VALUES ('Pooja Shah', 20, 'BCA')";
        int rows = stmt.executeUpdate(sql);
        System.out.println(rows + " record(s) inserted successfully.");

        stmt.close(); conn.close();
    }
}
```

Output: 1 record(s) inserted successfully.

Q12. Java program using JDBC to update existing records in a database table using a Statement.

```
import java.sql.*;
public class UpdateRecord {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college_db", "root", "password");
        Statement stmt = conn.createStatement();

        // Update age of a specific student
        String sql = "UPDATE students SET age = 25, course = 'MCA' " +
                     "WHERE name = 'Pooja Shah'";
        int rows = stmt.executeUpdate(sql);
        System.out.println(rows + " record(s) updated successfully.");
```

```
        stmt.close(); conn.close();
    }
}
```

Output: 1 record(s) updated successfully.

Q13. Java program using JDBC to delete records from a database table using a Statement.

```
import java.sql.*;
public class DeleteRecord {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college_db", "root", "password");
        Statement stmt = conn.createStatement();

        // Delete a specific student record
        String sql = "DELETE FROM students WHERE name = 'Pooja Shah'";
        int rows = stmt.executeUpdate(sql);
        System.out.println(rows + " record(s) deleted successfully.");

        stmt.close(); conn.close();
    }
}
```

Output: 1 record(s) deleted successfully.

Q14. Java program to demonstrate exception handling in JDBC while connecting to a database.

Concept:

JDBC can throw: **ClassNotFoundException** (driver not found), **SQLException** (DB errors). Use **try-catch-finally** to handle all cases and always close connections.

```
import java.sql.*;

public class JDBCExceptionHandling {
    public static void main(String[] args) {
        Connection conn = null;
        try {
            // This will throw ClassNotFoundException if driver jar is missing
            Class.forName("com.mysql.cj.jdbc.Driver");

            // This will throw SQLException if credentials are wrong
            conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college_db",
                "root",
                "wrongpassword"   // intentionally wrong
            );
            System.out.println("Connected!");

        } catch (ClassNotFoundException e) {
            System.out.println("[ERROR] JDBC Driver not found.");
            System.out.println("Cause: " + e.getMessage());
            System.out.println("Fix  : Add mysql-connector-java.jar to classpath.");

        } catch (SQLException e) {
            System.out.println("[ERROR] Database connection failed!");
            System.out.println("Error Code : " + e.getErrorCode());
            System.out.println("SQL State  : " + e.getSQLState());
            System.out.println("Message    : " + e.getMessage());

            // Handle specific error codes
            if (e.getErrorCode() == 1045)
                System.out.println("Reason: Wrong username or password.");
```

```
            else if (e.getErrorCode() == 1049)
                System.out.println("Reason: Database does not exist.");

        } finally {
            // Always close connection to release resources
            if (conn != null) {
                try {
                    conn.close();
                    System.out.println("Connection closed.");
                } catch (SQLException e) {
                    System.out.println("Error closing connection: " + e.getMessage());
                }
            }
        }
    }
}
```

Output (with wrong password):

```
[ERROR] Database connection failed!
Error Code : 1045
SQL State  : 28000
Message    : Access denied for user 'root'@'localhost' (using password: YES)
Reason     : Wrong username or password.
```

Q15. Java program using ResultSet to navigate data using next(), previous(), and first() methods.

Concept:

By default ResultSet is forward-only. To use **previous()** and **first()**, create a **scrollable** ResultSet using TYPE_SCROLL_INSENSITIVE.

```
import java.sql.*;

public class ResultSetNavigation {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college_db", "root", "password");

        // Create SCROLLABLE ResultSet
        Statement stmt = conn.createStatement(
            ResultSet.TYPE_SCROLL_INSENSITIVE,
            ResultSet.CONCUR_READ_ONLY);
        ResultSet rs = stmt.executeQuery("SELECT * FROM students");

        // ■■ next() ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        System.out.println("Using next():");
        while (rs.next())
            System.out.println("  " + rs.getInt("id") + " - " + rs.getString("name"));

        // ■■ first() ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        rs.first();
        System.out.println("
First record  : " + rs.getString("name"));

        // ■■ last() ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        rs.last();
        System.out.println("Last record   : " + rs.getString("name"));

        // ■■ absolute() ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        rs.absolute(2);
        System.out.println("2nd record    : " + rs.getString("name"));

        // ■■ previous() ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
        System.out.println("
Using previous() (backward):");
        rs.afterLast();
        while (rs.previous())
            System.out.println("  " + rs.getInt("id") + " - " + rs.getString("name"));

        // ■■ relative() ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        rs.first();
        rs.relative(2); // Move 2 rows forward from current
        System.out.println("
relative(+2) from first: " + rs.getString("name"));

        rs.close(); stmt.close(); conn.close();
    }
}
```

ResultSet Navigation Methods:

```
next()        - Move to next row          (returns false if no more rows)
previous()    - Move to previous row       (requires scrollable RS)
first()       - Move to first row
last()        - Move to last row
absolute(n)   - Move to row number n
relative(n)   - Move n rows from current position
beforeFirst() - Move before first row
afterLast()   - Move after last row
```

Q16. Java program using PreparedStatement to insert records into a database securely.

Concept:

PreparedStatement prevents SQL Injection attacks by using parameterized queries. User input is treated as data, not as SQL code.

```
import java.sql.*;
import java.util.Scanner;

public class SecureInsert {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college_db", "root", "password");

        // PreparedStatement with placeholders (?)
        String sql = "INSERT INTO students(name, age, course) VALUES (?, ?, ?)";
        PreparedStatement pstmt = conn.prepareStatement(sql);

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Name   : ");
        String name = sc.nextLine();

        System.out.print("Enter Age    : ");
        int age = Integer.parseInt(sc.nextLine());

        System.out.print("Enter Course : ");
        String course = sc.nextLine();

        // Set values - safe from SQL injection
        pstmt.setString(1, name);
        pstmt.setInt(2, age);
        pstmt.setString(3, course);

        int rows = pstmt.executeUpdate();
        System.out.println(rows + " record inserted securely!");

        pstmt.close(); conn.close();
    }
}
```

Why PreparedStatement is Secure:

```
// UNSAFE (Statement - SQL Injection possible):
String name = "'; DROP TABLE students; --"; // malicious input
stmt.executeUpdate("INSERT INTO students VALUES('" + name + "')");
// This would DELETE the entire table!

// SAFE (PreparedStatement):
pstmt.setString(1, name); // treated as literal data, not SQL
// The single quote in name is automatically escaped
```

■ *Note: Always use PreparedStatement when user input is involved in SQL queries.*

Q17. Java program using PreparedStatement to retrieve records based on user input.

```
import java.sql.*;
import java.util.Scanner;

public class SearchByInput {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college_db", "root", "password");

        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Search by Course (e.g., MCA/BCA/BSc CS): ");
        String course = sc.nextLine();

        // PreparedStatement for SELECT with user input
        String sql = "SELECT * FROM students WHERE course = ?";
        PreparedStatement pstmt = conn.prepareStatement(sql);
        pstmt.setString(1, course);

        ResultSet rs = pstmt.executeQuery();

        System.out.println("\nStudents in " + course + ":");
        System.out.println("+----+------------------+-----+");
        System.out.println("| ID | Name             | Age |");
        System.out.println("+----+------------------+-----+");

        boolean found = false;
        while (rs.next()) {
            found = true;
            System.out.printf("| %-2d | %-16s | %-3d |%n",
                rs.getInt("id"), rs.getString("name"), rs.getInt("age"));
        }
        if (!found)
            System.out.println("| No records found for: " + course + "  |");

        System.out.println("+----+------------------+-----+");

        rs.close(); pstmt.close(); conn.close();
    }
}
```

Output:

```
Search by Course (e.g., MCA/BCA/BSc CS): MCA

Students in MCA:
+----+------------------+-----+
| ID | Name             | Age |
+----+------------------+-----+
| 2  | Priya Patil      | 21  |
| 5  | Ravi More        | 21  |
+----+------------------+-----+
```

Q18. Java program to demonstrate the use of different types of JDBC drivers.

Theory — 4 Types of JDBC Drivers:

Type	Name	Description	Example
Type 1	JDBC-ODBC Bridge Driver	Converts JDBC calls to ODBC calls. Requires ODBC driver. Deprecated in Java 8.	sun.jdbc.odbc.JdbcOdbcDriver
Type 2	Native-API Driver	Uses native (C/C++) DB client libraries. Platform dependent.	Oracle OCI Driver
Type 3	Network Protocol Driver	Sends JDBC calls to middleware server which translates to DB protocol.	IDS Driver
Type 4	Thin Driver (Pure Java)	Directly converts JDBC calls to DB-specific protocol. Most commonly used. 100% Java.	com.mysql.jdbc.Driver

Program demonstrating Type 4 Driver (most widely used):

```
import java.sql.*;

public class JDBCDriverTypes {
    public static void main(String[] args) {
        // TYPE 4: Pure Java / Thin Driver for MySQL
        String url  = "jdbc:mysql://localhost:3306/college_db";
        String user = "root";
        String pass = "password";

        try {
```

```
            // Load Type 4 MySQL Driver (automatically loaded in JDBC 4.0+)
            Class.forName("com.mysql.cj.jdbc.Driver");
            System.out.println("Type 4 Driver: MySQL Connector/J loaded.");

            Connection conn = DriverManager.getConnection(url, user, pass);
            DatabaseMetaData meta = conn.getMetaData();

            System.out.println("Driver Name    : " + meta.getDriverName());
            System.out.println("Driver Version : " + meta.getDriverVersion());
            System.out.println("JDBC Version   : " + meta.getJDBCMajorVersion()
                + "." + meta.getJDBCMinorVersion());
            System.out.println("DB Product     : " + meta.getDatabaseProductName());
            System.out.println("DB Version     : " + meta.getDatabaseProductVersion());

            conn.close();
        } catch (ClassNotFoundException e) {
            System.out.println("Driver not found: " + e.getMessage());
        } catch (SQLException e) {
            System.out.println("SQL Error: " + e.getMessage());
        }
    }
}
```

Output:

```
Type 4 Driver: MySQL Connector/J loaded.
Driver Name    : MySQL Connector/J
Driver Version : mysql-connector-java-8.0.30
JDBC Version   : 4.2
DB Product     : MySQL
DB Version     : 8.0.30
```

Q19. Java program using CallableStatement to execute a stored procedure from Java.

Concept:

CallableStatement is used to call stored procedures stored in the database. Syntax: **{call procedure_name(?, ?)}**. Supports IN, OUT, and INOUT parameters.

Step 1: Create Stored Procedures in MySQL:

```
-- Procedure 1: IN parameter only
DELIMITER //
CREATE PROCEDURE GetStudentByID(IN stud_id INT)
BEGIN
    SELECT id, name, age, course FROM students WHERE id = stud_id;
END //

-- Procedure 2: IN and OUT parameter
CREATE PROCEDURE GetStudentCount(IN course_name VARCHAR(30), OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total FROM students WHERE course = course_name;
END //

DELIMITER ;
```

Step 2: Java Program using CallableStatement:

```
import java.sql.*;

public class CallableStatementDemo {
    static final String URL  = "jdbc:mysql://localhost:3306/college_db";
    static final String USER = "root";
    static final String PASS = "password";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```
            // ■■ CALL Procedure with IN parameter ■■■■■■■■■■■■
            System.out.println("=== Call GetStudentByID ===");
            CallableStatement cs1 = conn.prepareCall("{call GetStudentByID(?)}");
            cs1.setInt(1, 2);  // Set IN parameter
            ResultSet rs = cs1.executeQuery();
            while (rs.next())
                System.out.println("ID: " + rs.getInt("id") +
                    " | Name: "   + rs.getString("name") +
                    " | Age: "    + rs.getInt("age") +
                    " | Course: " + rs.getString("course"));
            rs.close(); cs1.close();

            // ■■ CALL Procedure with IN and OUT parameter ■■■■■
            System.out.println("\n=== Call GetStudentCount ===");
            CallableStatement cs2 = conn.prepareCall("{call GetStudentCount(?, ?)}");
            cs2.setString(1, "MCA");          // IN param
            cs2.registerOutParameter(2, Types.INTEGER); // Register OUT param
            cs2.execute();
            int count = cs2.getInt(2);        // Retrieve OUT value
            System.out.println("Total MCA students: " + count);
            cs2.close();

        } catch (ClassNotFoundException e) {
            System.out.println("Driver not found: " + e.getMessage());
        } catch (SQLException e) {
            System.out.println("SQL Error [" + e.getErrorCode() + "]: " + e.getMessage());
        }
    }
}
```

Output:

```
=== Call GetStudentByID ===
ID: 2 | Name: Priya Patil | Age: 21 | Course: MCA

=== Call GetStudentCount ===
Total MCA students: 3
```

■ *Note: Use registerOutParameter() to register OUT parameters before calling execute(). Then retrieve the value using getInt(), getString(), etc.*